МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 4

з дисципліни "Професійний Python"

на тему "Професійне ООП та система типів"

Виконав
студент 3 курсу
групи ФЕП-32
Козачок Ростислав Михайлович

Перевірив
Куньо Іван Михайлович

Львів – 2026

Мета роботи: Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

Хід роботи:

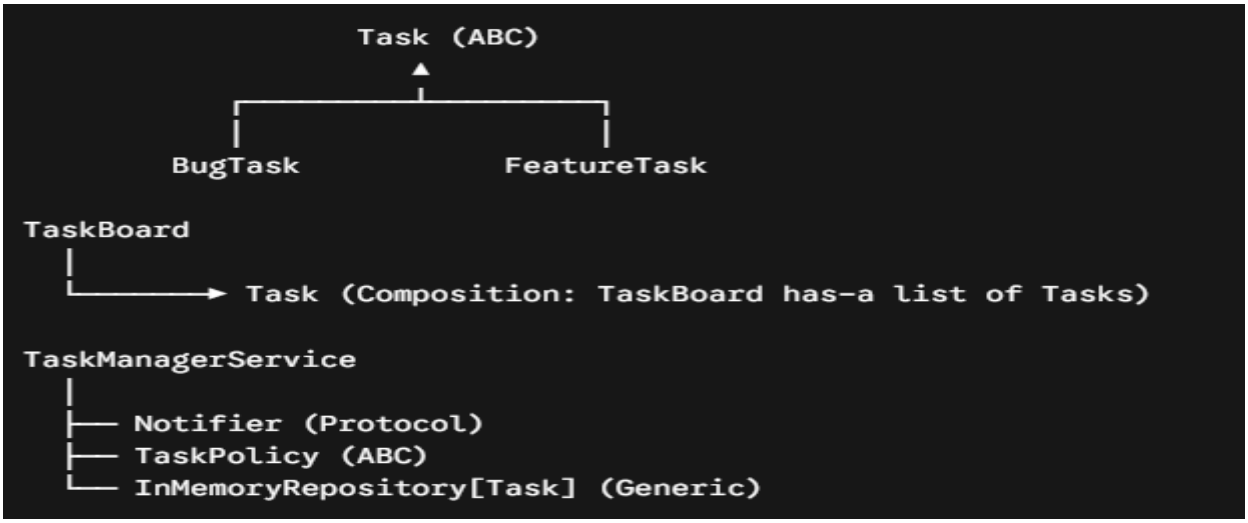
Варіант 14. Система керування завданнями

Предметна область описує систему керування завданнями (Task Management System), яка дозволяє створювати завдання різних типів (баги, фічі), відстежувати їхній статус, оцінювати витрати часу та управляти ними на віртуальній дошці.

Список Domain Entities:

1. Task (Абстрактне базове завдання)
2. BugTask (Завдання типу "помилка")
3. FeatureTask (Завдання типу "нова функція")
4. TaskBoard (Дошка завдань)
5. TaskPolicy (Політика/правила зміни станів завдань)

Діаграма класів:



Архітектурні та ООП рішення:

- **Наслідування (Inheritance):** Використано для відношення is-a. BugTask та FeatureTask успадковують базові поля від Task (id, title, priority), розширюючи їх специфічними атрибутами (наприклад, steps_to_reproduce для багів). Це відповідає Liskov Substitution Principle.
- **Композиція (Composition):** Використано для відношення has-a. Клас TaskBoard не успадковується від вбудованого list, а містить приватний список завдань self._tasks: list[Task]. Це дозволяє інкапсулювати логіку роботи з дошкою.

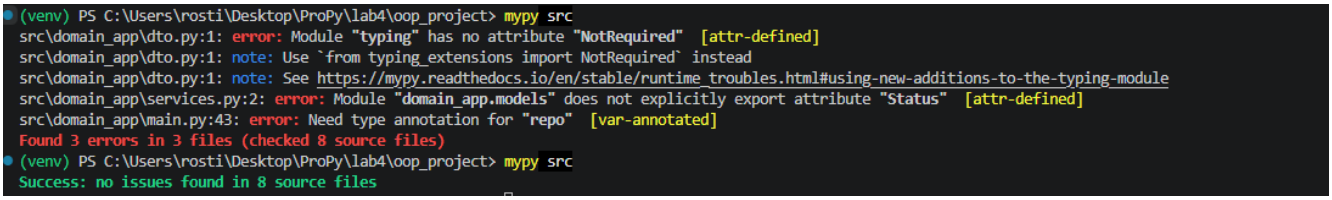
- **Інкапсуляція (Encapsulation):** Приватне поле self._items у InMemoryRepository та self._tasks у TaskBoard приховані від прямого доступу ззовні. Робота з ними ведеться виключно через публічні методи (add, get, all).
- **Property:** У класі Task використано @property def is_completed(self) -> bool, що надає зручний інтерфейс для перевірки статусу (без необхідності викликати метод з дужками), приховуючи логіку перевірки.
- **Dataclass:** Базовий клас Task та його нащадки позначені як @dataclass(kw_only=True). Це автоматично генерує __init__, __repr__ та __eq__, зменшуючи кількість шаблонного коду.
- **Immutable Value Object:** EffortEstimate реалізовано як @dataclass(frozen=True, slots=True). Об'єкт оцінки часу є незмінним. У разі потреби додати час, створюється новий екземпляр (через метод __add__).
- **Abstract Base Class (ABC):** Task є абстрактним класом (успадковує ABC). Також створено абстракцію TaskPolicy з методом @abstractmethod def can_close(...), що зобов'язує конкретні політики реалізувати цю поведінку.
- **Protocol:** Створено Notifier(Protocol). Будь-який клас, що має метод send(recipient, message), автоматично відповідає цьому протоколу (Structural Subtyping). Це дозволяє не прив'язувати сервіс до конкретної реалізації розсилки.
- **Dunder methods:**
    1. __add__ (в EffortEstimate) — для додавання оцінок часу.
    2. __len__ (в TaskBoard) — для отримання кількості завдань через len(board).
    3. __iter__ (в TaskBoard) — для можливості ітерації по дошці for task in board.
    4. __contains__ (в TaskBoard) — для перевірки наявності завдання task_id in board.
- **Generic та TypeVar:** Створено узагальнений репозиторій InMemoryRepository[T], де T = TypeVar("T", bound=HasId). Це дозволяє перевикористовувати один клас репозиторію для будь-яких сутностей, що мають ідентифікатор.
- **TypedDict:** Реалізовано TaskPayload для типізації словників із зовнішніми даними (наприклад, JSON з API), де поле severity вказано як NotRequired[str].
- **Application Service:** TaskManagerService оркеструє бізнес-процес: отримує завдання з репозиторію, валідує зміну статусу через TaskPolicy, оновлює стан та відправляє сповіщення через Notifier.
- **Dependency Injection (DI):** Залежності repository, notifier та policy

передаються в конструктор TaskManagerService ззовні. Це повністю відв'язує сервіс від конкретних реалізацій і полегшує тестування.

Принципи SOLID:

1. **SRP (Single Responsibility Principle):** Кожен клас має одну зону відповідальності. TaskBoard відповідає лише за групування завдань, InMemoryRepository — за збереження в пам'яті, ConsoleNotifier — за вивід повідомлень.
2. **OCP (Open/Closed Principle):** Клас TaskManagerService закритий для модифікацій, але відкритий для розширення. Щоб змінити правила закриття завдань, достатньо передати нову реалізацію TaskPolicy, не змінюючи код самого сервісу.
3. **DIP (Dependency Inversion Principle):** TaskManagerService залежить не відконкретного ConsoleNotifier чи InMemoryRepository, а від абстрактних інтерфейсів Notifier та узагальненого сховища. Залежності спрямовані на абстракції.

Аналіз статичної типізації:



Основний код програми:

```
from domain_app.value_objects import Priority, Status, EffortEstimate
from domain_app.models import BugTask, FeatureTask, TaskBoard
from domain_app.repositories import InMemoryRepository
from domain_app.services import TaskManagerService, ConsoleNotifier, StrictTaskPolicy
from domain_app.models import Task

def main() -> None:
    bug = BugTask(
        id=1,
        title="Fix login screen crash",
        description="App crashes on empty password",
        priority=Priority.CRITICAL,
        estimate=EffortEstimate(2.5),
        steps_to_reproduce="1. Open App 2. Click Login",
        severity="High"
    )

    feature = FeatureTask(
        id=2,
        title="Add OAuth2 Login",
```

```
        description="Support Google/GitHub login",
        priority=Priority.MEDIUM,
        estimate=EffortEstimate(10.0),
        business_value=100
    )

    board = TaskBoard("Sprint 1")
    board.add_task(bug)
    board.add_task(feature)

    print(f"Total tasks on board: {len(board)}") # __len__
    print(f"Is task 1 on board? {1 in board}")   # __contains__

    print("\n--- Board Tasks ---")
    for task in board: # __iter__
        print(task)    # __str__

    print("\n--- Stats ---")
    print(board.stats)

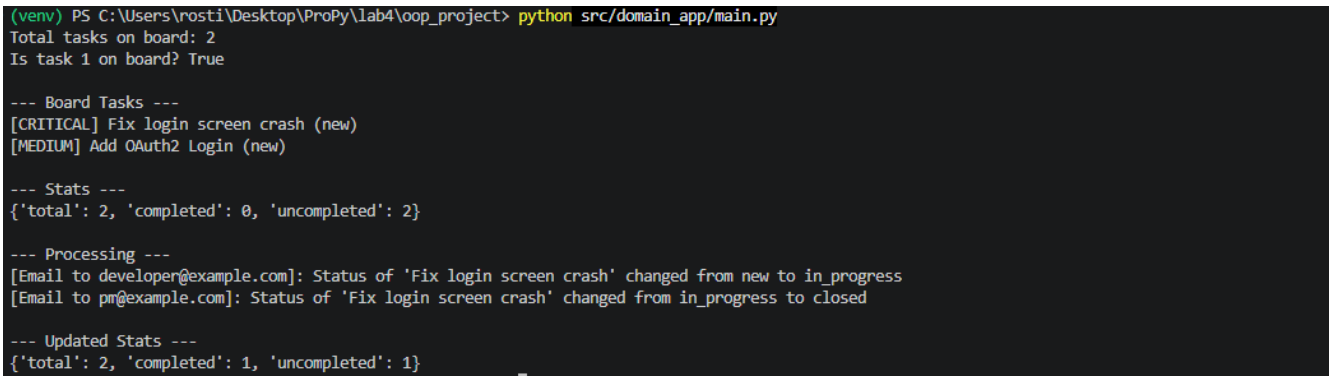
    repo = InMemoryRepository[Task]()
    repo.add(bug)
    repo.add(feature)

    service = TaskManagerService(
        repository=repo,
        notifier=ConsoleNotifier(),
        policy=StrictTaskPolicy()
    )
    print("\n--- Processing ---")
    # Переводимо баг в In Progress, а потім в Closed
    service.change_task_status(1, Status.IN_PROGRESS, "developer@example.com")
    service.change_task_status(1, Status.CLOSED, "pm@example.com")

    print("\n--- Updated Stats ---")
    print(board.stats) # Статистика оновиться, бо об'єкти передаються за посиланням

if __name__ == "__main__":
    main()
```

Скріншот виконання:



Висновок:

Під час виконання лабораторної роботи було створено повноцінну об'єктну модель системи керування завданнями. Перехід від процедурного стилю до професійного ООП забезпечив модульність системи: бізнес-логіка відділена від інфраструктури (репозиторіїв та сповіщень) завдяки Dependency Injection та протоколам. Використання статичного аналізатора mypy у строгому режимі дозволило виявити приховані проблеми типізації (неявні імпорти, відсутність типізації дженериків) ще до запуску програми, що підтверджує важливість сучасної системи type hints у Python для побудови надійних систем.